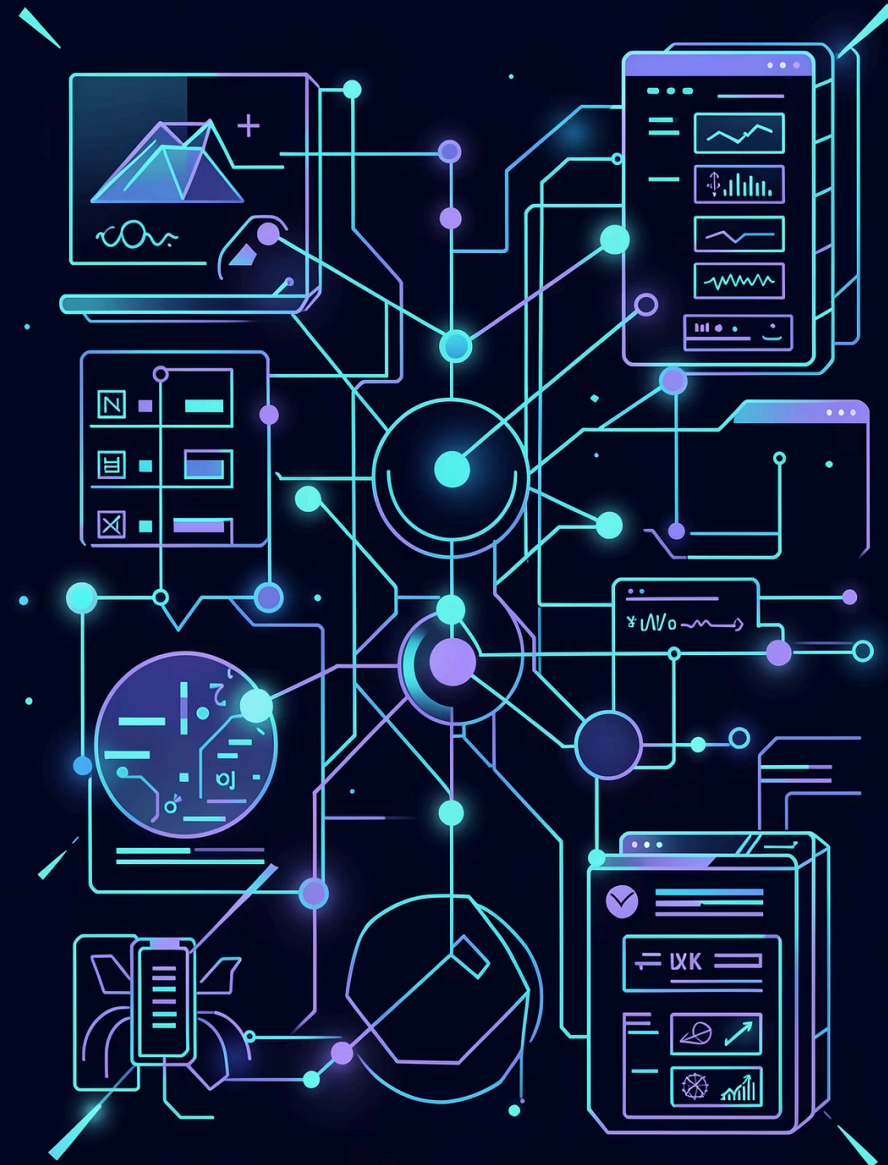


Arrays, Matrices y Listas Enlazadas

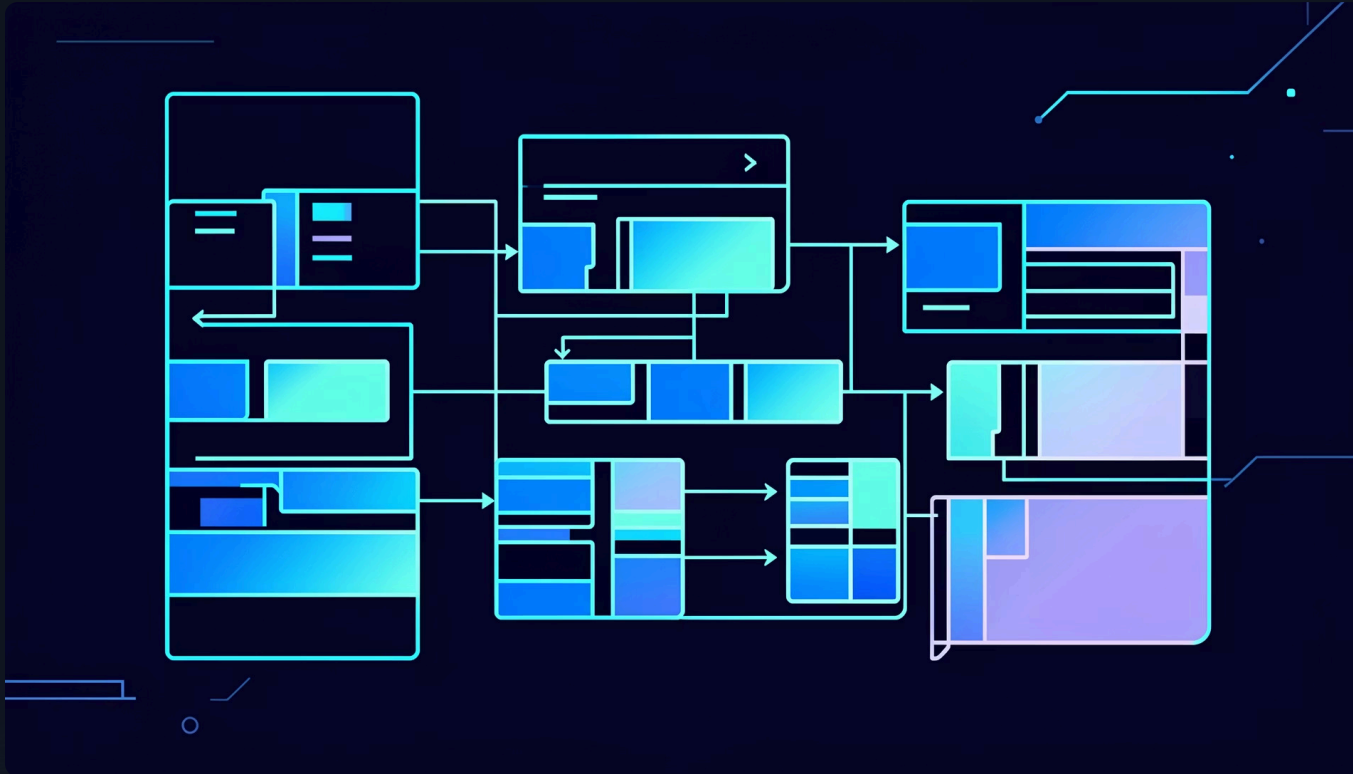
Estructuras de datos fundamentales en desarrollo de software

ASIGNATURA: DESARROLLO DE SOFTWARE



Made with GAMMA

Concepto de Array



Un **array** es una estructura de datos estática que almacena elementos del mismo tipo en posiciones consecutivas de memoria. Cada elemento es accesible mediante un índice numérico que determina su posición relativa.

Almacenamiento contiguo

Los elementos ocupan bloques consecutivos en la memoria RAM

Acceso indexado

Acceso directo mediante índice en tiempo constante $O(1)$

Tamaño fijo

La capacidad se define en tiempo de compilación

Concepto de Matriz

Definición técnica

Una **matriz** es un array multidimensional que organiza datos en una estructura tabular de filas y columnas. Representa una extensión del concepto de array a dos o más dimensiones.

- ❏ Las matrices son especialmente útiles para representar datos bidimensionales como imágenes digitales, tablas de datos o sistemas de coordenadas cartesianas.

Características principales

- Estructura bidimensional o n-dimensional
- Organización mediante sistema de coordenadas (fila, columna)
- Memoria contigua en disposición lineal
- Acceso mediante múltiples índices

Concepto de Lista Enlazada

Una **lista enlazada** es una estructura de datos dinámica formada por nodos distribuidos en memoria de forma no contigua. Cada nodo contiene un dato y una referencia (puntero) al siguiente nodo de la secuencia.

01

Nodos independientes

Cada nodo se almacena en ubicaciones arbitrarias de memoria

02

Referencias dinámicas

Los punteros conectan los nodos formando la secuencia lógica

03

Asignación dinámica

La memoria se solicita en tiempo de ejecución según necesidad



Diferencias en la Gestión de Memoria



La disposición física en memoria determina el rendimiento y las características operacionales de cada estructura.

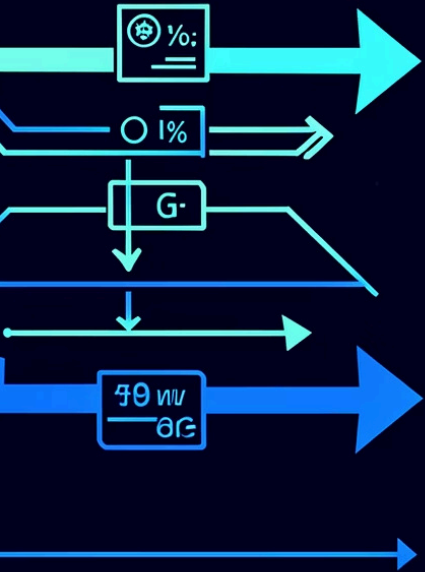
Arrays y Matrices

- Bloques contiguos de memoria
- Asignación estática en tiempo de compilación
- Acceso predictivo mediante aritmética de punteros
- Mayor aprovechamiento de caché del procesador

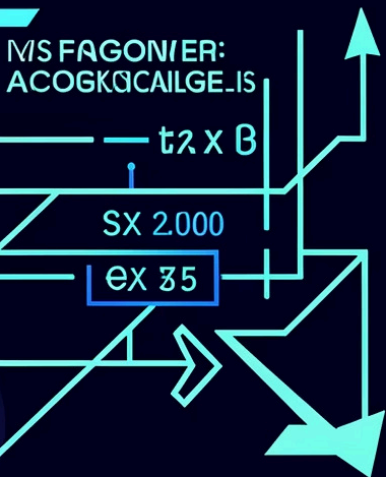
Listas Enlazadas

- Nodos dispersos en memoria heap
- Asignación dinámica en tiempo de ejecución
- Acceso mediante seguimiento de referencias
- Overhead adicional por almacenar punteros

DIRECT ACCESS

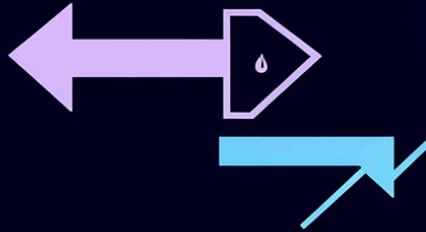


DIREASET ACCESS



SEQUENTIAL ACCESS

- VOSOT - CRG13,9 2Y
- EOCF,3 V = $\frac{270}{x}$ ARB 1E



DATA STRUCTUREE

- $ec7.33 = 00.E\ 1\ 139$
 $R1: 18a53$
- $Ac6106X = A1, E2X3$



Diferencias en Acceso y Modificación

El acceso a cualquier elemento se realiza en **tiempo constante $O(1)$** mediante cálculo directo de la dirección de memoria. La fórmula $\text{base_address} + (\text{índice} \times \text{tamaño_elemento})$ permite localización inmediata sin recorridos intermedios.

El acceso requiere **tiempo lineal $O(n)$** en el peor caso, ya que es necesario recorrer la cadena de nodos desde el inicio hasta alcanzar la posición deseada, siguiendo las referencias sucesivas entre elementos.

Arrays y matrices tienen **tamaño inmutable** tras su declaración. Las listas enlazadas permiten **crecimiento dinámico** mediante inserción y eliminación de nodos en tiempo constante $O(1)$ en posiciones conocidas.

Ventajas y Limitaciones



Eficiencia en Acceso

Arrays/Matrices:
Óptimos para lectura frecuente y acceso aleatorio

Listas Enlazadas:
Eficientes en inserciones y eliminaciones frecuentes



Flexibilidad Estructural

Arrays/Matrices:
Estructura rígida, tamaño predefinido

Listas Enlazadas:
Crecimiento dinámico, adaptabilidad en ejecución

Impacto en el Rendimiento

La selección de la estructura de datos adecuada afecta directamente la eficiencia temporal y espacial del algoritmo:

- **Localidad de referencia:** Arrays aprovechan la caché del CPU
- **Overhead de memoria:** Listas requieren espacio adicional para punteros
- **Complejidad algorítmica:** Operaciones básicas varían entre $O(1)$ y $O(n)$
- **Fragmentación:** Listas pueden causar fragmentación del heap

Cuadro Comparativo Conceptual

Análisis multidimensional de las tres estructuras de datos fundamentales según criterios técnicos relevantes para el diseño de algoritmos.

Criterio	Array	Matriz	Lista Enlazada
Gestión de Memoria	Contigua, estática	Contigua, estática multidimensional	No contigua, dinámica
Tipo de Acceso	Directo $O(1)$	Directo $O(1)$ con índices múltiples	Secuencial $O(n)$
Complejidad Inserción	$O(n)$ en posiciones intermedias	$O(n)$ requiere desplazamiento	$O(1)$ con referencia al nodo
Complejidad Eliminación	$O(n)$ en posiciones intermedias	$O(n)$ requiere reorganización	$O(1)$ con referencia al nodo
Overhead de Memoria	Mínimo	Mínimo	Alto (punteros adicionales)
Aplicaciones Típicas	Buffers, vectores matemáticos	Imágenes, tablas, gráficos 3D	Pilas, colas, listas dinámicas

Criterios de Selección



Analizar Requisitos

Frecuencia de operaciones y patrones de acceso



Evaluar Complejidad

Costo temporal y espacial de operaciones críticas



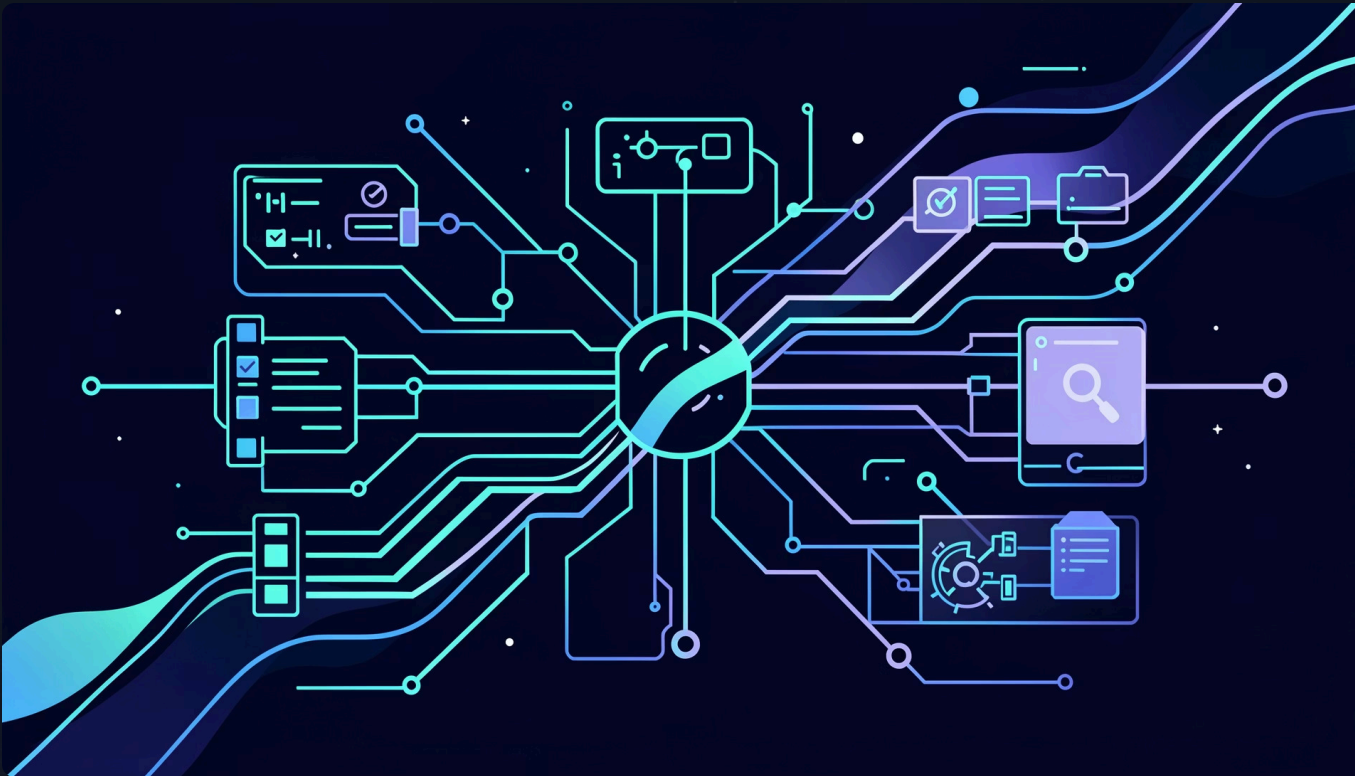
Seleccionar Estructura

Equilibrio óptimo entre eficiencia y funcionalidad

La elección adecuada de la estructura de datos es fundamental para el diseño de soluciones computacionalmente eficientes y escalables en el desarrollo de software profesional.



Conclusiones Clave



1 No existe una estructura universalmente óptima

La selección depende del contexto y requisitos específicos del problema

2 El análisis de complejidad es determinante

Comprender el costo de operaciones guía decisiones de diseño

3 La memoria y el rendimiento están interrelacionados

Optimizar uno puede comprometer el otro según la estructura elegida